

# **EnergyForecast AI**

## **Cloud-Deployed LSTM System for Next-Day WTI Oil Price Forecasting**

Applied AI Solutions Development

AASD 4016 / 4017

George Brown College

**Kourosh Shahram, PhD**

## 1. Executive Summary

EnergyForecast AI is an end-to-end machine-learning forecasting system that predicts the next trading day's WTI crude oil price. The project extends beyond model training by integrating live market-data acquisition, feature engineering, an LSTM neural network, API-based inference, Docker containerization, cloud deployment, and a Streamlit user interface.

The deployed system automatically retrieves recent market information, constructs 26 engineered features, selects the most recent 60 complete observations, and sends the resulting 60 x 26 matrix to a FastAPI prediction service hosted in a Docker container on Render. The trained LSTM returns the forecast as JSON, and Streamlit displays the result to the user.

This forecasting system is also intended to be used as part of a larger Optimal Energy Resource Planning project. The current project focuses on the forecasting component as a functional machine-learning and deployment system.

## 2. Project Objective and Scope

The main objective was to transform a trained oil-price forecasting model into a publicly accessible cloud-based application. A model that works only in a notebook or on a local computer has limited operational value. The project therefore addresses the complete path from historical data and model training to live inference through a web-accessible service.

- Predict the next trading day's WTI crude oil price in USD per barrel.
- Use a 60-day sequential input window and 26 engineered market features.
- Automate live data retrieval and feature preparation so no manual 60-row input is required.
- Expose the trained model through a FastAPI web API.
- Package the inference environment with Docker and deploy it on Render.
- Provide a Streamlit interface that retrieves live data and displays the forecast.

## 3. Machine-Learning Task

The problem is a supervised time-series regression task. The target is a continuous numerical value: the next trading day's WTI crude oil price.

| Element           | Definition                                   |
|-------------------|----------------------------------------------|
| Input             | 60 consecutive days x 26 engineered features |
| Output            | One next-day WTI oil-price forecast          |
| Problem type      | Supervised time-series regression            |
| Lookback          | 60 days                                      |
| Primary framework | TensorFlow / Keras                           |

## 4. Why LSTM?

Oil prices form sequential time-series data. The ordering of observations matters because current market conditions can be related to patterns that developed over previous days and weeks. Long Short-Term Memory (LSTM) networks are designed to process sequences and learn temporal dependencies.

The model reads 60 consecutive days before generating one next-day prediction. LSTM memory mechanisms allow the network to learn which information from the sequence is useful to retain, update, or forget. This makes LSTM a suitable architecture for the project's sequential forecasting problem.

## 5. Data Sources and Data Collection

The system uses external market-data sources for both historical model development and live inference.

- FRED API: WTI crude oil, natural gas, U.S. Dollar Index, and VIX.
- yfinance: gold market data.

For model development, the project uses approximately 4,933 daily records covering 2006-2026.

The live pipeline fetches the market data, aligns and cleans it, engineers all required features, removes incomplete rows, and selects the most recent 60 fully engineered observations.

## 6. Feature Engineering

The LSTM does not rely only on the oil price. The system transforms raw market data into 26 engineered features designed to represent market levels, recent history, momentum, volatility, and derived relationships.

- Market levels: oil, gold, natural gas, USD Index, and VIX.
- Lag and momentum variables: 1-, 5-, 10-, and 21-day lags and 7-/21-day momentum.
- Rolling statistics: 7-, 21-, and 60-day rolling measures, means, standard deviations, and volatility.
- Derived signals: log returns, Gold/Oil ratio, VIX spike indicator, and calendar information.

A key engineering requirement is consistency between training and serving. The shared feature pipeline is used by both processes so that live inputs are transformed in the same way as training data, reducing train/serve skew.

## 7. Model Architecture and Training

The forecasting network contains two LSTM layers followed by dense layers that convert the learned sequential representation into one numerical price prediction.

Architecture: LSTM(128) -> Dropout(0.2) -> LSTM(64) -> Dropout(0.2) -> Dense(32) -> Dense(1).

The first LSTM layer contains 128 units and the second contains 64 units. A dropout rate of 0.2 is used during training to reduce overfitting by randomly dropping 20% of layer outputs. The model uses the Adam optimizer and mean squared error loss. EarlyStopping and ReduceLROnPlateau callbacks are used during training.

The data is divided using an 80/20 chronological split. Preserving chronological order is important because random shuffling could allow information from later periods to influence training on earlier periods.

## 8. Model Evaluation

Offline evaluation is performed on historical test data that was not used for model fitting. The reported results are:

| Metric | Result |
|--------|--------|
|--------|--------|

|       |                        |
|-------|------------------------|
| MAPE  | 2.97%                  |
| $R^2$ | 0.928                  |
| MAE   | approximately USD 2.35 |

These metrics describe historical test performance. They provide evidence of predictive capability but should not be interpreted as a guarantee of future market accuracy.

## 9. System Architecture

The production system is divided between two managed cloud services connected through a JSON API contract.

- Streamlit Community Cloud: user interface, live data retrieval, feature engineering, construction of the 60 x 26 input matrix, and forecast display.
- Render: Dockerized FastAPI inference service that loads the trained LSTM and returns the prediction.
- JSON POST /predict: communication mechanism between Streamlit and the deployed inference service.

This separation keeps raw market-data handling outside the Dockerized inference service. The Render service receives the finished model input rather than fetching raw market data itself.

## 10. API, Uvicorn, Docker, and Render

FastAPI defines the web endpoints used to access Python prediction functionality. Uvicorn acts as the ASGI server: it listens for incoming network requests and passes them to the FastAPI application.

Docker packages the application code, TensorFlow/Keras environment, trained LSTM model, saved scaler, and prediction services into a reproducible container. On Render, this container runs in the cloud and exposes the inference service.

The main prediction flow is: client request -> Render -> Docker container -> Uvicorn -> FastAPI -> scaler/model service -> LSTM inference -> JSON response.

## 11. Automated Live Prediction Workflow

1. Fetch approximately 500 recent days of market data from FRED and yfinance.
2. Clean, align, and engineer the 26 required features.
3. Select the most recent 60 complete engineered rows.
4. Create the 60 x 26 model input matrix.
5. Send the matrix as JSON in an HTTP POST request to the FastAPI /predict endpoint.
6. Apply the saved preprocessing/scaling logic required by the trained model.
7. Run LSTM inference.
8. Convert the prediction to the required oil-price scale and return it as JSON.
9. Display the forecast in the Streamlit application.

This workflow replaced an earlier approach that required manual entry of 60 rows of market data. The deployed version therefore provides zero-manual-input forecasting from live data.

## 12. Deployment Challenges and Solutions

### 12.1 Data Reliability

- FRED chart-export requests were unreliable in cloud environments. The solution was to use FRED's official documented REST API with explicit timeout handling.
- Earlier gold-data sources proved unstable. They were replaced with yfinance.
- Docker DNS lookups could hang before normal HTTP request timeouts applied. Network/DNS handling was adjusted to improve reliability.

### 12.2 Environment and Dependency Compatibility

- TensorFlow compatibility constrained the Python environment, requiring a supported Python version.
- Pinned NumPy dependencies created build issues when compatible pre-built wheels were unavailable; compatible NumPy and scikit-learn versions were selected.
- Free-tier hosting may suspend inactive containers, causing the first request after idle time to incur a cold-start delay.
- The FRED API key is managed as a secret/environment variable and is not hardcoded in committed source code.

## 13. Live Evaluation and Monitoring

After deployment, forecasts should be compared with the subsequently observed WTI prices. Model performance can be tracked over time using MAE, RMSE, MAPE, and  $R^2$  over appropriate evaluation windows.

Technical monitoring should include API and /health availability, FRED and yfinance retrieval, missing observations, failed predictions, response times, and Render/Streamlit service availability. Significant deterioration in prediction accuracy or changes in feature distributions can indicate model or data drift and can trigger offline retraining and reevaluation.

## 14. Decision Support and Broader Project Context

The next-day WTI forecast can support short-term energy planning, budgeting, procurement, risk assessment, and market analysis. The system is designed as decision support rather than as an autonomous trading mechanism.

This forecasting system is also intended to be used as part of a larger Optimal Energy Resource Planning project. In that broader context, the oil-price forecast can provide an input to subsequent resource-planning and optimization activities.

## 15. Conclusion

EnergyForecast AI demonstrates the transition from a trained time-series model to an operational cloud-based machine-learning system. The project combines historical and live market data, feature engineering, LSTM forecasting, FastAPI, Uvicorn, Docker, Render, and Streamlit into one end-to-end workflow.

The key achievement is not only the forecast itself, but the integration required to make the model accessible and usable outside the development environment. The system automatically prepares live data, serves

predictions through a public API, and provides a foundation for integration into the broader Optimal Energy Resource Planning initiative.

## Appendix A. Machine Learning Canvas Summary

| Canvas Element                 | Project Answer                                                                                                                                                                                                                                                                    |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Decisions                      | The predicted next-day WTI oil price can support short-term decisions related to energy planning, budgeting, procurement, risk assessment, and market analysis. This forecasting system is also intended to be used as part of a larger Optimal Energy Resource Planning project. |
| ML Task                        | Input: latest 60 days x 26 features. Output: next trading day's WTI price. Type: supervised time-series regression.                                                                                                                                                               |
| Value Propositions             | Automated, accessible next-day forecasting without manual market-data preparation or local model execution.                                                                                                                                                                       |
| Data Sources                   | FRED API for WTI, natural gas, USD Index and VIX; yfinance for gold.                                                                                                                                                                                                              |
| Collecting Data                | Retrieve recent market data, align and engineer features, and add new observations for future evaluation/retraining.                                                                                                                                                              |
| Making Predictions             | On demand: fetch -> engineer -> latest 60 rows -> JSON POST -> FastAPI -> LSTM -> JSON forecast -> Streamlit.                                                                                                                                                                     |
| Offline Evaluation             | 80/20 chronological split; MAPE 2.97%, $R^2$ 0.928, MAE about USD 2.35.                                                                                                                                                                                                           |
| Features                       | 26 market, lag, momentum, rolling-statistic, volatility, return, ratio, VIX and calendar features.                                                                                                                                                                                |
| Building Models                | Two-layer LSTM with dropout and dense output layers; Adam/MSE; EarlyStopping and ReduceLROnPlateau.                                                                                                                                                                               |
| Live Evaluation and Monitoring | Compare forecasts with actual WTI prices; monitor MAE/RMSE/MAPE/ $R^2$ , data drift, API health, data sources, failures and latency.                                                                                                                                              |